

Computergrafik

PD Prof. Dr. rer nat.
Marco Block-Berlitz

Sommersemester 2026



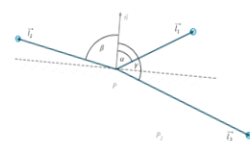
Grundlagen der Vektorgeometrie

„Was wir sehen, ist niemals die Wirklichkeit selbst, sondern eine Interpretation.“ (Rudolf Arnheim)

PD Prof. Dr. Marco Block-Berlitz

Vektoren liegen also in drei Varianten vor

Vektoren als **Richtungen**
 $\vec{v}, \vec{w}$

$$\cos \alpha = \frac{\vec{v}^T \vec{w}}{\|\vec{v}\| \cdot \|\vec{w}\|}$$


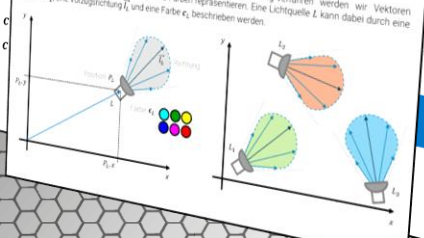
Vektoren als **Positionen**
 L, P, Q

$$F = \frac{1}{2} \| (P_2 - P_1) \times (P_3 - P_1) \|$$

Vektoren als **Farben**
 c, d

Notation einer Lichtquelle

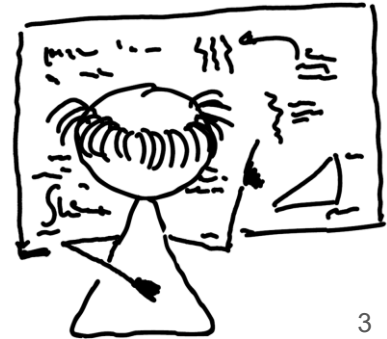
Für die übersichtlichere Darstellung von Beleuchtungs- und Rendering-Verfahren werden wir Vektoren unterschieden: die Positionen, Richtungen und Farben repräsentieren. Eine Lichtquelle L kann dabei durch eine Position P_L , eine Vorzugsrichtung $\vec{t}_L$ und eine Farbe c_L beschrieben werden.



33

Ziele und Inhalt

- Sichtbares Licht und Strahlenoptik
- Sichtbares Licht
- Fokus zunächst auf Strahlenoptik
- Notation einer Lichtquelle
- Vektoren und deren Operationen in Java
- Vektoren als Richtungen
- Vektoren als Richtungen und Positionen
- Funktion `length` versus `lengthSquare`
- Kosinusformel und Skalarprodukt
- Kosinusformel Herleitung
- Übertragung auf rechtwinkliges Dreieck
- Sohcahtoa und ein wichtiges Gesetz des Kosinus
- Beweis zum Gesetz des Kosinus
- Kosinusformel Umstellung und Normierung
- Konventionen zur Werteeinschränkung
- Sinusformel nach der Herleitung
- Skalarprodukt und Halbräume
- Einheitskreis und Normierung eines Vektors
- Normierung eines Vektors
- Winkelangaben in Bogenmaß oder Radiant
- Winkelangaben umwandeln
- Vektoren als Farben
- Vektoren liegen also in drei Varianten vor
- Fragestellungen zum Vorlesungsteil
- Praktische Aufgaben



Sichtbares Licht und Strahlenoptik

Zunächst wird es für uns hilfreich sein, Licht als **Lichtstrahlen** anzusehen (Strahlenoptik). Wir haben also eine Lichtquelle, die solche Lichtstrahlen in geeigneter Weise abgibt.

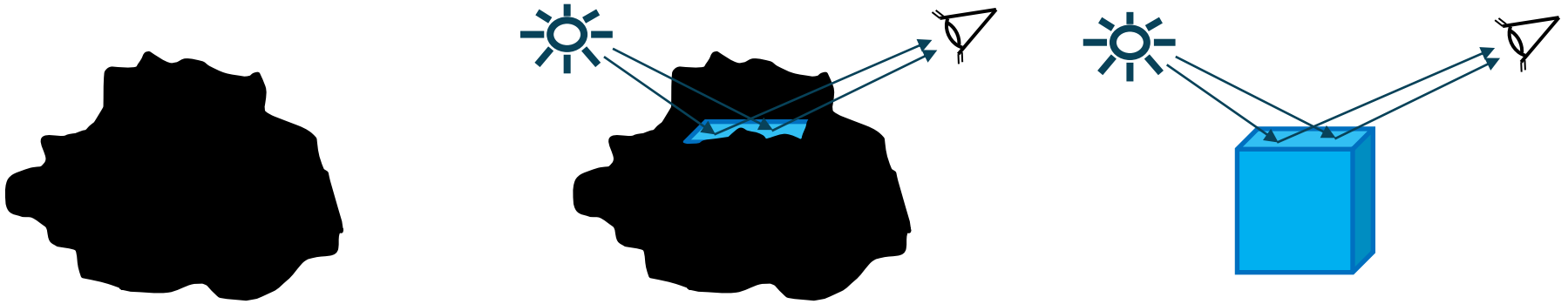




Warum sehen wir eigentlich Objekte?

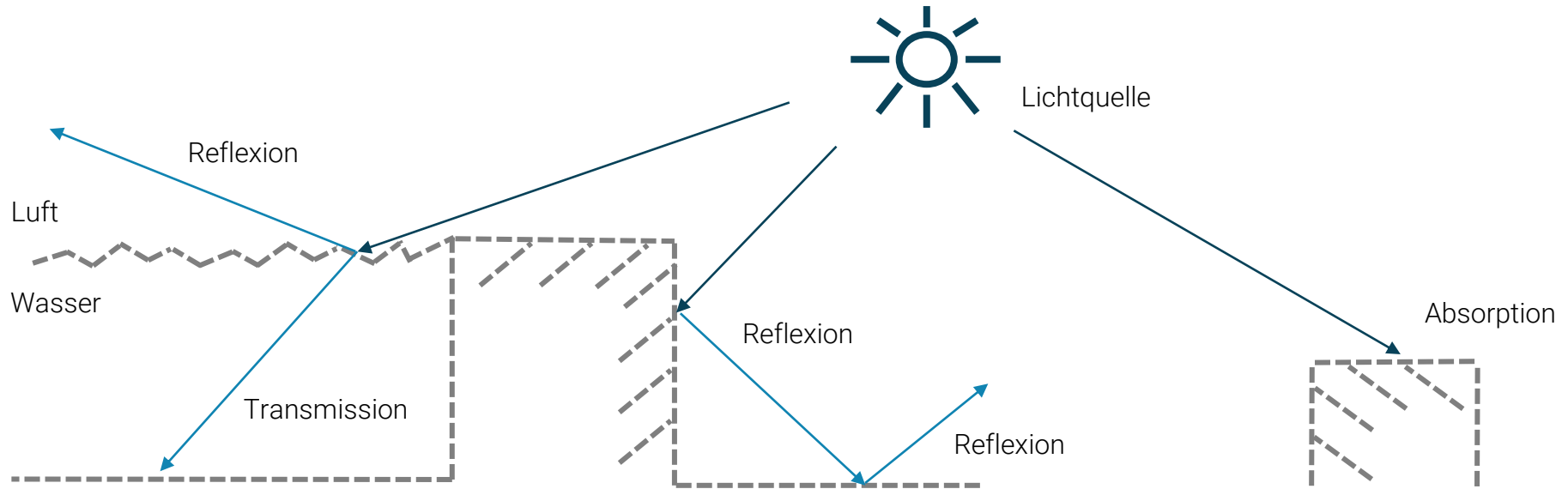
Sichtbares Licht

Lichtstrahlen werden von der Oberfläche eines Objekts **reflektiert** und erreichen anschließend unser Auge. Gilt unter der typischen Annahme: Lichttransport im Vakuum, d. h. Lichtinteraktion findet nur an den Oberflächen der Objekte und nicht in der Umgebung statt.



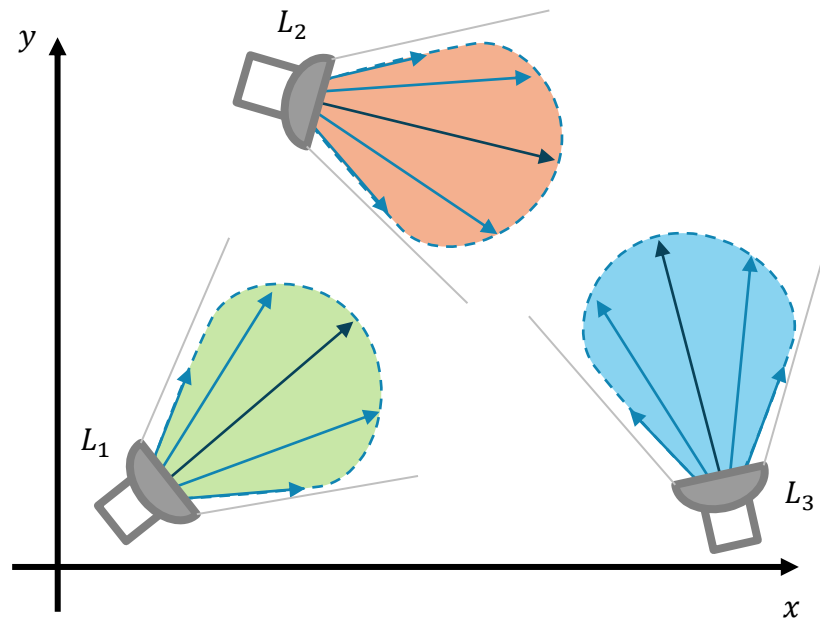
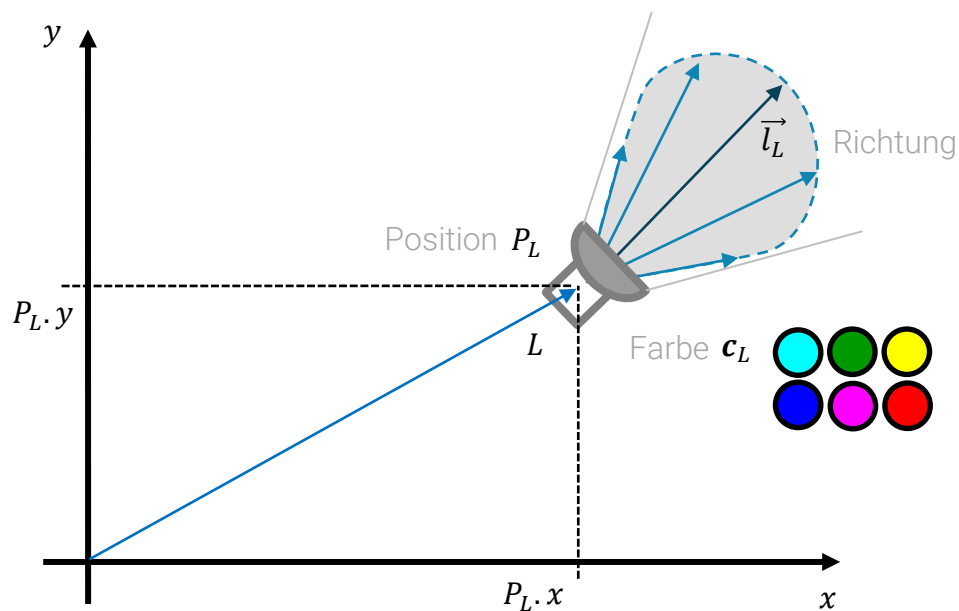
Fokus zunächst auf Strahlenoptik

Die **Strahlenoptik** wird auch als geometrische Optik bezeichnet und stellt die Strahleneigenschaften von Licht in den Vordergrund. Damit lassen sich bereits einfache Modelle und Erklärungen für die Lichtbrechung und die Reflexion finden.



Notation einer Lichtquelle

Für die übersichtlichere Darstellung von Beleuchtungs- und Rendering-Verfahren werden wir Vektoren unterscheiden, die Positionen, Richtungen und Farben repräsentieren. Eine Lichtquelle L kann dabei durch eine Position P_L , eine Vorzugsrichtung $\vec{l}_L$ und eine Farbe $\mathbf{c}_L$ beschrieben werden.



Vektoren und deren Operationen in Java

Für das bessere Verständnis zu den Vektoroperationen wollen wir die Java-Klassen **Vektor2D**, **Vektor3D** und **LineareAlgebra** hier einführen:

```
public class Vektor2D {
    public double x, y;

    public Vektor2D() {
        this(0, 0);
    }

    public Vektor2D(double x, double y) {
        setX(x);
        setY(y);
    }

    public Vektor2D(Vektor2D vec) {
        this(vec.x, vec.y);
    }
    ...
}
```

Java

```
public class Vektor3D {
    public double x, y, z;
    ...
}
```

Java

```
public class LineareAlgebra {
    private LineareAlgebra() {};

    public static Vektor2D add(Vektor2D vec1, Vektor2D vec2) {
        return new Vektor2D(vec1.x+vec2.x, vec1.y+vec2.y);
    }
    ...
}
```

Java

Hier sehen wir ein Anwendungsbeispiel, das den Unterschied zwischen beiden Varianten verdeutlichen soll (**mutable** versus **immutable**):

```
public static void main(String[] args) {
    Vektor2D a = new Vektor2D(5, 3);
    Vektor2D b = new Vektor2D(1, 1);
    a.add(b);
    a.show();
    Vektor2D c = LineareAlgebra.add(a, b);
    c.show();
}
```

Java

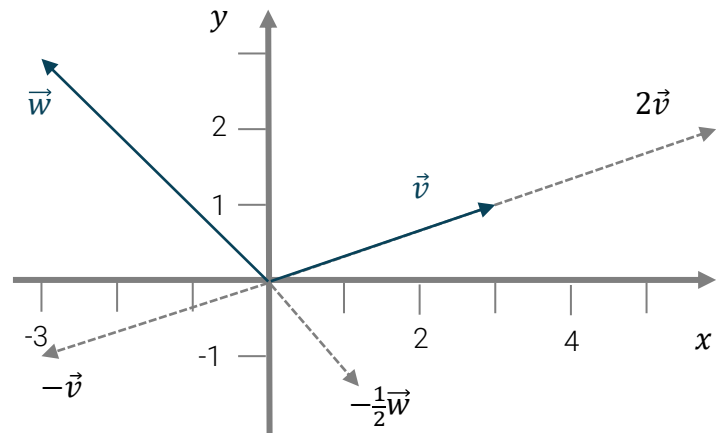
(6.0, 4.0)
(7.0, 5.0)



Vektoren als Richtungen

Die bekannte Vektorschreibweise $\vec{v}$ werden wir für Vektoren verwenden, die Richtungen repräsentieren. Es handelt sich dabei immer um **Spaltenvektoren**:

$$\vec{v} = \begin{bmatrix} v_1 \\ v_2 \\ \vdots \\ v_n \end{bmatrix} \quad \vec{v} = (v_1, v_2, \dots, v_n)$$



In **LineareAlgebra** können wir auf die Klassen **Vektor2D** und **Vektor3D** zurückgreifen:

```
public static Vektor3D mult(Vektor3D vec, double s) {  
    return new Vektor3D(vec.x*s, vec.y*s, vec.z*s);  
}  
  
public static Vektor3D mult(double s, Vektor3D vec) {  
    return mult(vec, s);  
}
```

Java

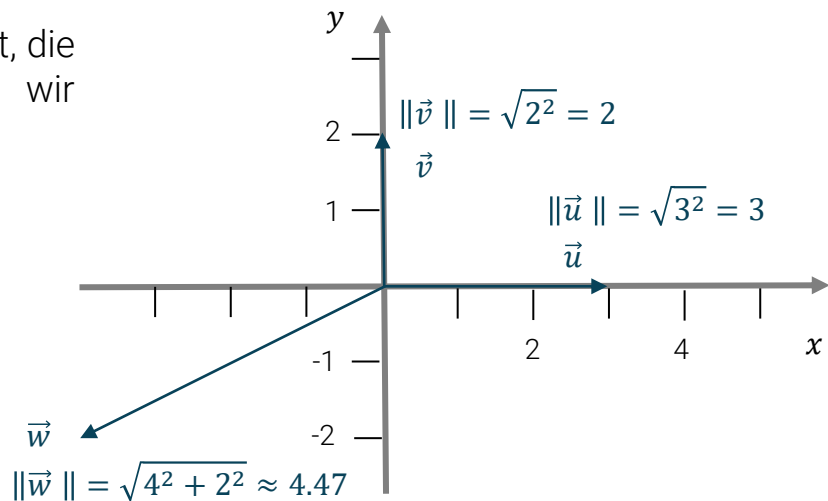
Vektoren als Richtungen und Positionen

Die Notation von Punkten L, P, Q wird für Vektoren verwendet, die **Positionen** repräsentieren, z. B. $A = (2,4)$. Dabei können wir jederzeit zwischen Punkt- und Vektordarstellungen wechseln:

$$\text{dist}_{EUK}(A, B) = \|A - B\| = \text{length}(\vec{a} - \vec{b})$$

Die **Länge eines Vektors** $\vec{v}$ lässt sich wie folgt bestimmen:

$$\text{length}(\vec{v}) = \|\vec{v}\| = \sqrt{v_1^2 + v_2^2 + \dots + v_n^2}$$



Bei der Implementierung gibt es allerdings eine **Besonderheit**.



Funktion length versus lengthSquare

Da in vielen Fällen die Länge eines Vektors für den Vergleich mit anderen Vektorlängen verwendet wird, um daraufhin eine Entscheidung zu treffen, genügt es dann, eine schnellere Variante zu verwenden, bei der auf die rechenintensive Funktion **Math.sqrt** verzichtet wird:

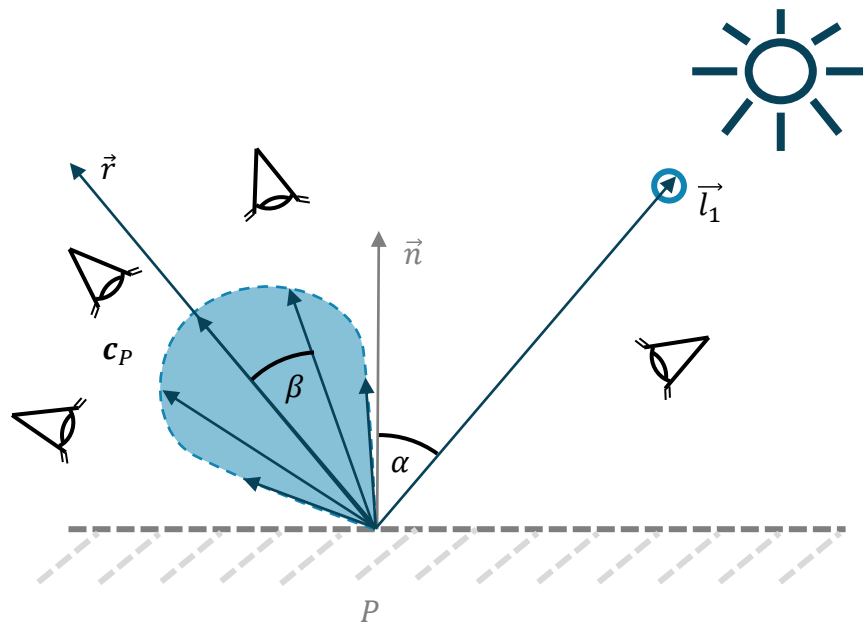
```
public static double length(Vektor3D vec) {  
    return Math.sqrt(lengthSquare(vec));  
}  
  
public static double lengthSquare(Vektor3D vec) {  
    return vec.x*vec.x + vec.y*vec.y + vec.z*vec.z;  
}
```

Java

Die **Monotonie** bleibt erhalten, da folgende Äquivalenzrelation gilt:

$$\text{length}(\vec{v}) > \text{length}(\vec{w}) \Leftrightarrow \text{lengthSquare}(\vec{v}) > \text{lengthSquare}(\vec{w})$$

Oft ist es ratsam, nach solchen **effizienten Teillösungen** Ausschau zu halten.





Kommen wir zu der vermutlich wichtigsten Formel der Computergrafik (**Kosinusformel**):

$$\cos \alpha = \frac{\vec{v} \cdot \vec{w}}{\|\vec{v}\| \cdot \|\vec{w}\|}$$

Kosinusformel und Skalarprodukt

Über die [Kosinusformel](#) können wir den Kosinus des Winkels zwischen zwei Vektoren ermitteln. Dazu brauchen wir zunächst das [innere Produkt](#) (auch Punkt- oder Skalarprodukt genannt):

$$\vec{v} \cdot \vec{w} = \sum_{i=1}^n v_i \cdot w_i \quad \vec{v}^T \vec{w} = \sum_{i=1}^n v_i \cdot w_i$$

```
public static double dotProduct(Vektor3D vec1, Vektor3D vec2) {  
    return vec1.x*vec2.x + vec1.y*vec2.y + vec1.z*vec2.z;  
}
```

Java

Jetzt können wir daraus die Kosinusformel ableiten:

$$\cos \alpha = \frac{\vec{v} \cdot \vec{w}}{\|\vec{v}\| \cdot \|\vec{w}\|}$$

```
public static double kosinusFormel(Vektor3D vec1, Vektor3D vec2) {  
    return dotProduct(vec1, vec2) / (vec1.length()*vec2.length());  
}
```

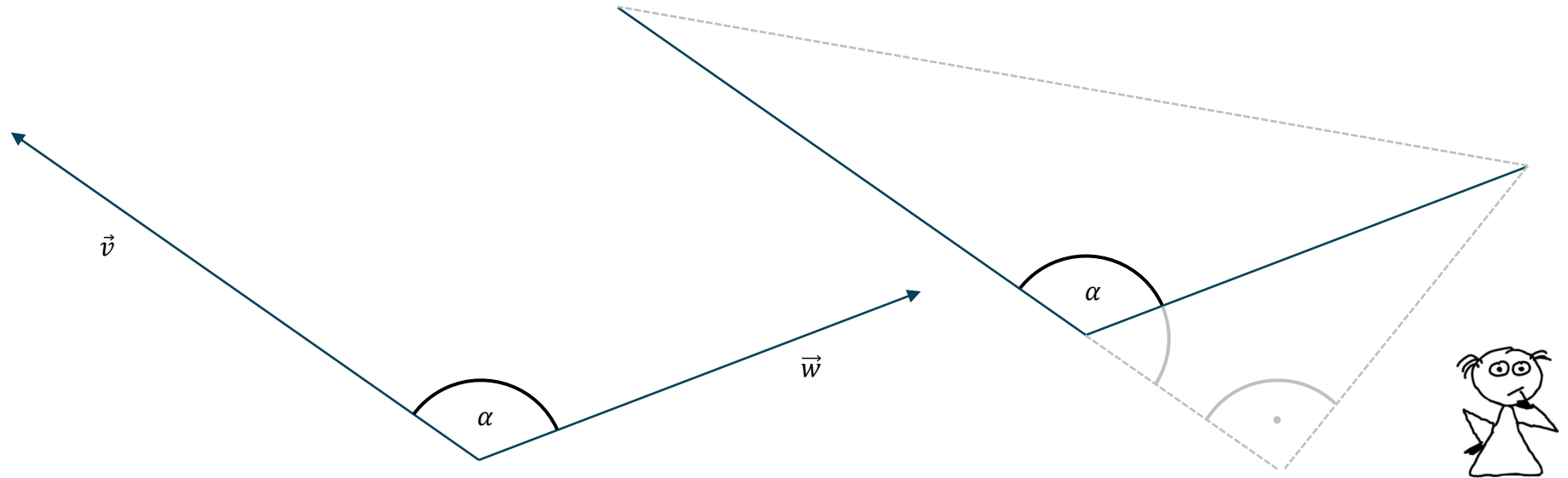
Java



Die **Kosinusformel** ist so wichtig, dass wir sie gleich mal gemeinsam herleiten wollen.

Kosinusformel Herleitung

Wir fragen uns, wie wir den Winkel α zwischen zwei gegebenen Vektoren $\vec{v}$ und $\vec{w}$ berechnen können. Wenn wir den Zusammenhang der Vektoren durch ein rechtwinkliges Dreieck erweitern, können wir mit Hilfe trigonometrischer Funktionen eine Antwort auf die Frage finden.



Übertragung auf rechtwinkliges Dreieck

Pythagoras lehrte uns den wichtigen Zusammenhang:

$$x^2 + y^2 = w^2 \qquad y^2 = w^2 - x^2$$

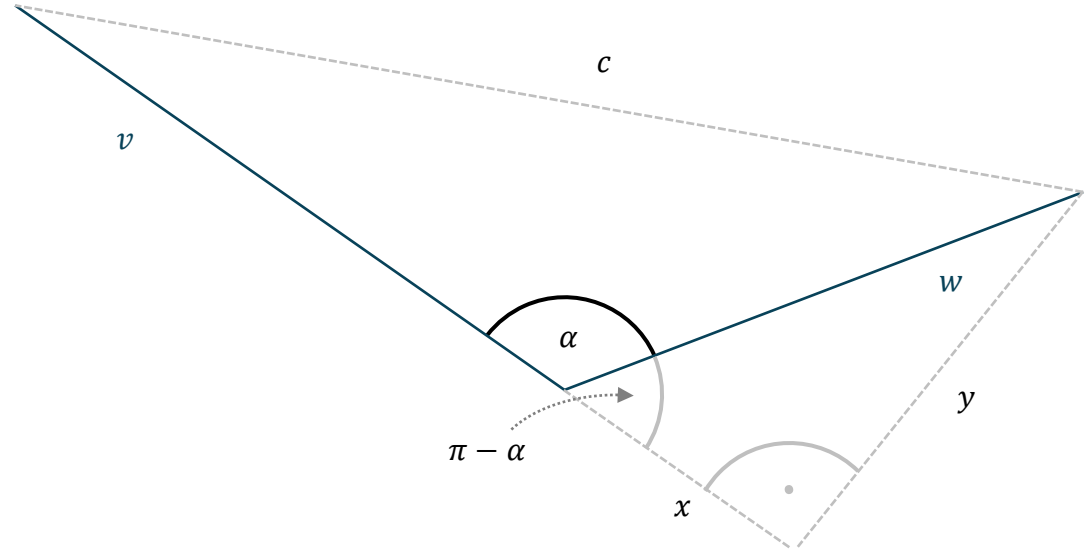
Mit Hilfe dessen können wir eine wichtige Gleichung ableiten und gleich einen der beiden unbekannten Werte ersetzen können:

$$(v + x)^2 + y^2 = c^2$$

$$(v + x)^2 + w^2 - x^2 = c^2$$

$$v^2 + 2vx + x^2 + w^2 - x^2 = c^2$$

$$v^2 + w^2 + 2vx = c^2$$



Sohcahtoa und ein wichtiges Gesetz des Kosinus

„Sohcahtoa“, der Gott der Winkel hilft uns hier weiter:

$$\cos(\pi - \alpha) = \frac{x}{w} \quad \cos(\pi - \alpha) = -\cos \alpha \quad x = -w \cos \alpha$$

Hier stehen wir bisher (y konnte bereits reduziert werden) und jetzt können wir endlich auch das freie x entfernen:

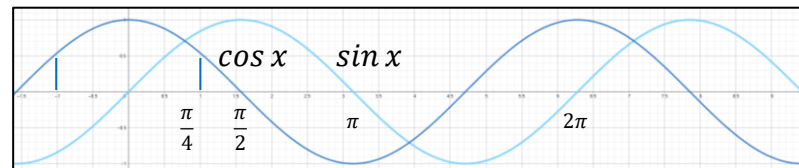
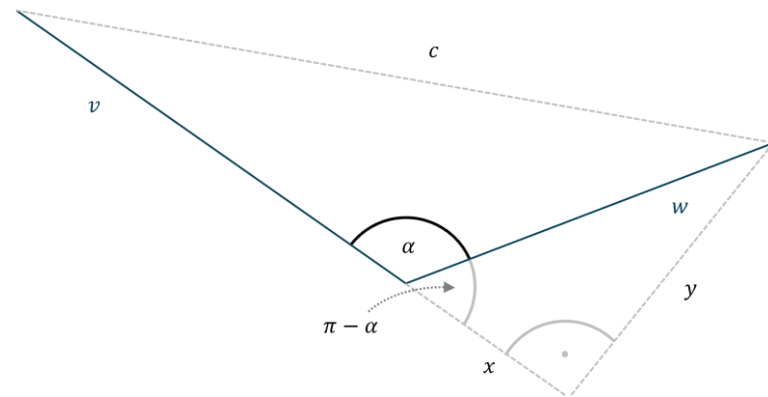
$$v^2 + w^2 + 2vx = c^2$$

Wenn wir das jetzt in unsere erste Erkenntnis einsetzen

$$v^2 + w^2 - 2vw \cos \alpha = c^2$$

erhalten wir ein wichtiges Gesetz des Kosinus!

Wir sehen jetzt schon, dass bei einem rechten Winkel der Term $2vw \cos \alpha$ zu 0 wird, woraus direkt der Satz des Pythagoras folgt.



Beweis zum Gesetz des Kosinus

Theorem Gegeben seien zwei n -dimensionale Vektoren $\vec{v}$ und $\vec{w}$, dann erfüllt das innere Produkt die folgende Gleichung:

$$\vec{v}^T \cdot \vec{w} = \|\vec{v}\| \cdot \|\vec{w}\| \cdot \cos \alpha$$

Beweis Aus dem Gesetz des Kosinus wissen wir

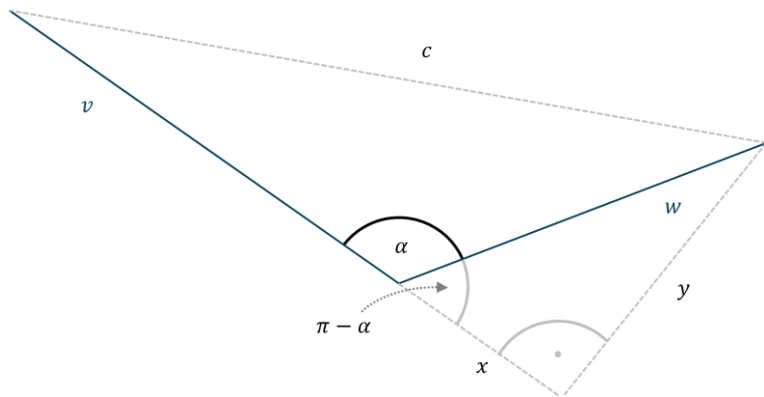
$$\|\vec{v} - \vec{w}\|^2 = \|\vec{v}\|^2 + \|\vec{w}\|^2 - 2\|\vec{v}\|\|\vec{w}\| \cos \alpha$$

$$\sum_{i=1}^n (v_i - w_i)^2 = \sum_{i=1}^n v_i^2 + \sum_{i=1}^n w_i^2 - 2\|\vec{v}\|\|\vec{w}\| \cos \alpha$$

$$\sum_{i=1}^n -2v_i w_i = -2\|\vec{v}\|\|\vec{w}\| \cos \alpha$$

$$\sum_{i=1}^n v_i w_i = \|\vec{v}\|\|\vec{w}\| \cos \alpha$$

$$\vec{v}^T \vec{w} = \|\vec{v}\|\|\vec{w}\| \cos \alpha$$



$$v^2 + w^2 - 2vw \cos \alpha = c^2$$

Kosinusformel Umstellung und Normierung

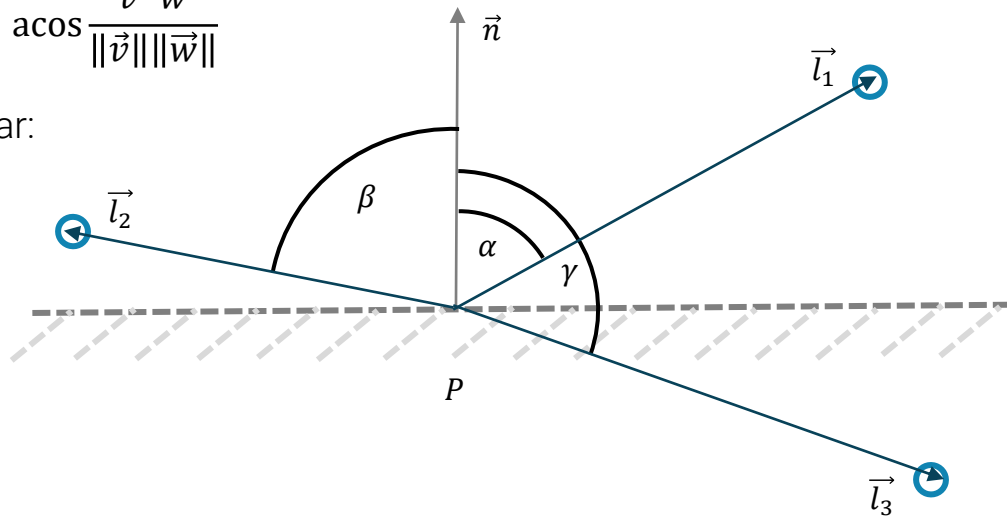
Jetzt ist es leicht, unsere **Kosinusformel** umzuformen:

$$\vec{v}^T \vec{w} = \|\vec{v}\| \|\vec{w}\| \cos \alpha \quad \cos \alpha = \frac{\vec{v}^T \vec{w}}{\|\vec{v}\| \|\vec{w}\|} \quad \alpha = \arccos \frac{\vec{v}^T \vec{w}}{\|\vec{v}\| \|\vec{w}\|}$$

Wenn beide Vektoren **normiert vorliegen**, gilt sogar:

$$\cos \alpha = \vec{v}^T \vec{w}$$

Da die Winkel zwischen die für uns relevanten Vektoren $\vec{n}$, $\vec{v}$, $\vec{r}$, $\vec{t}$ bzw. $\vec{l}$ ständig benötigt werden, achten wir darauf, dass diese **immer normiert** vorliegen.



Konventionen zur Werteeinschränkung

Hier sehen wir wichtige Konventionen zur Notation in dieser Veranstaltung:

$$\max\left(\frac{\vec{v}^T \vec{w}}{\|\vec{v}\| \|\vec{w}\|}, 0\right)$$

$$\begin{aligned} \max(\vec{v}^T \vec{w}, 0) \\ \min(a, 1) \end{aligned}$$

$$\underline{\vec{v}^T \vec{w}} = \max(\vec{v}^T \vec{w}, 0)$$

$$\overline{a} = \min(a, 1)$$

Im Programmcode findet sich dazu oft eine Methode `clamp`:

```
public static double clamp(double x, double min, double max) {  
    if (x < min)  
        return min;  
    else if (x > max)  
        return max;  
    return x;  
}
```

Java



Es gibt auch die **Sinusformel**. Die Herleitung wird eine der Übungsaufgaben sein.

Sinusformel nach der Herleitung

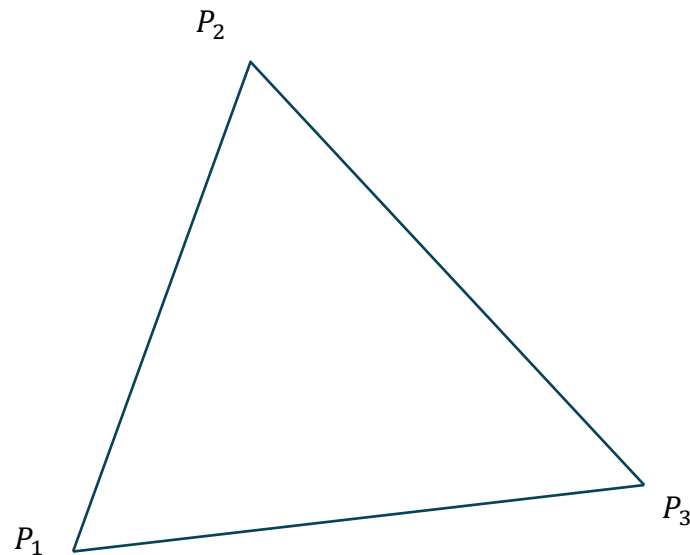
Hier sehen wir die Sinusformel analog zur Kosinusformel:

$$\|\vec{v} \times \vec{w}\| = \|\vec{v}\| \|\vec{w}\| \sin \alpha \quad \sin \alpha = \frac{\|\vec{v} \times \vec{w}\|}{\|\vec{v}\| \|\vec{w}\|} \quad \alpha = \arcsin \frac{\|\vec{v} \times \vec{w}\|}{\|\vec{v}\| \|\vec{w}\|}$$

Wenn beide Vektoren wieder normiert vorliegen, gilt:

$$\sin \alpha = \|\vec{v} \times \vec{w}\|$$

Wie könnten wir die Sinusformel nutzen, um die Fläche eines Dreiecks zu berechnen?



Sinusformel nach der Herleitung

Hier sehen wir die **Sinusformel** analog zur Kosinusformel:

$$\|\vec{v} \times \vec{w}\| = \|\vec{v}\| \|\vec{w}\| \sin \alpha \quad \sin \alpha = \frac{\|\vec{v} \times \vec{w}\|}{\|\vec{v}\| \|\vec{w}\|} \quad \alpha = \arcsin \frac{\|\vec{v} \times \vec{w}\|}{\|\vec{v}\| \|\vec{w}\|}$$

Wenn beide Vektoren wieder normiert vorliegen, gilt:

$$\sin \alpha = \|\vec{v} \times \vec{w}\|$$

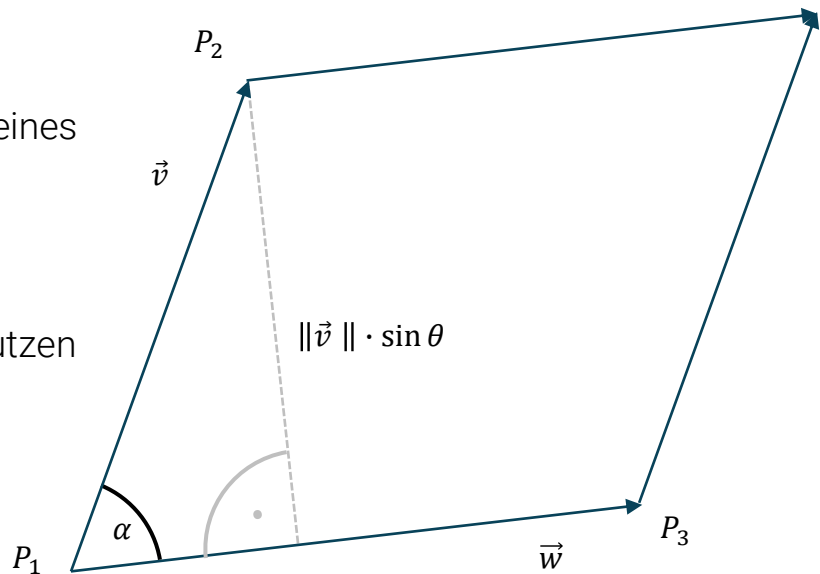
Wir können mit Hilfe der Sinusformel jetzt leicht die Fläche F eines Dreiecks $\Delta P_1 P_2 P_3$ angeben:

$$F = \frac{1}{2} \|\vec{v} \times \vec{w}\|$$

Die **Hälfte des Kreuzproduktes** können wir für das Ergebnis nutzen und konstruieren uns nur noch zwei passende Vektoren:

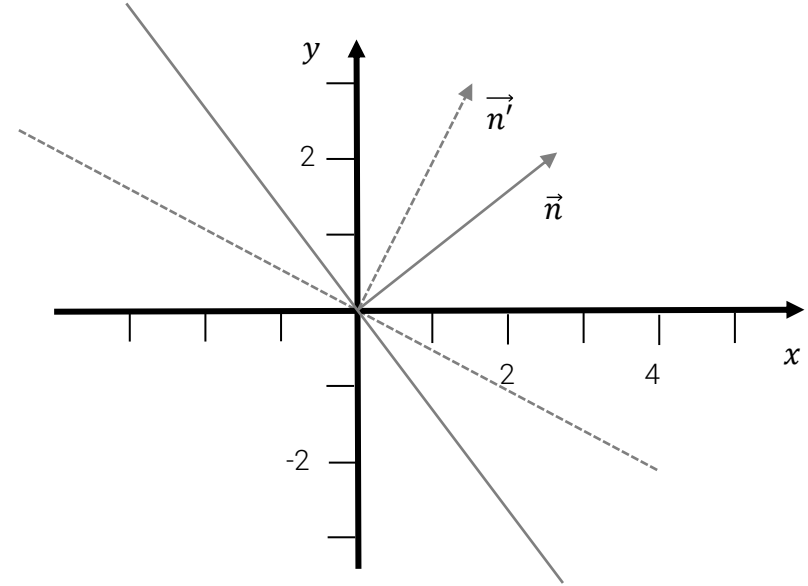
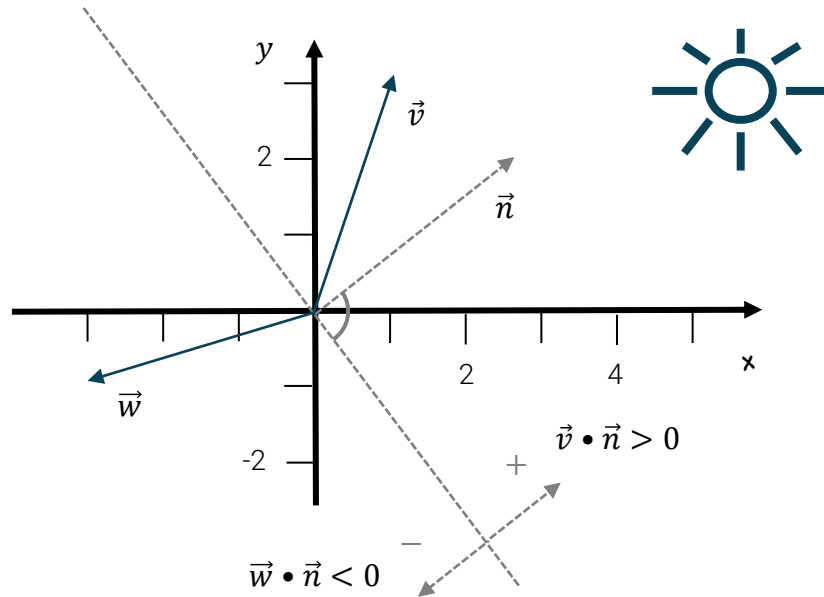
$$F = \frac{1}{2} \|(\mathbf{P}_2 - \mathbf{P}_1) \times (\mathbf{P}_3 - \mathbf{P}_1)\|$$

Wie könnten wir die Sinusformel nutzen, um die Fläche eines Dreiecks zu berechnen?



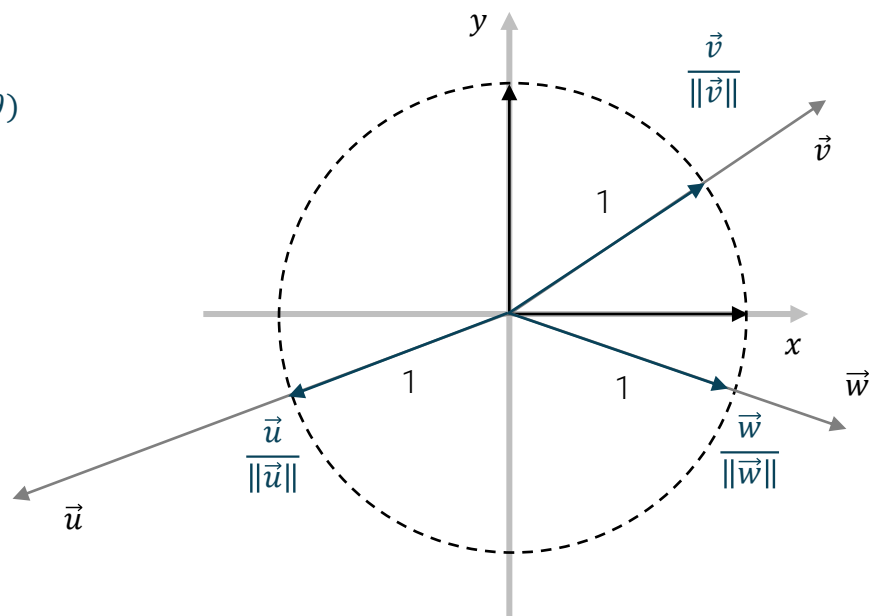
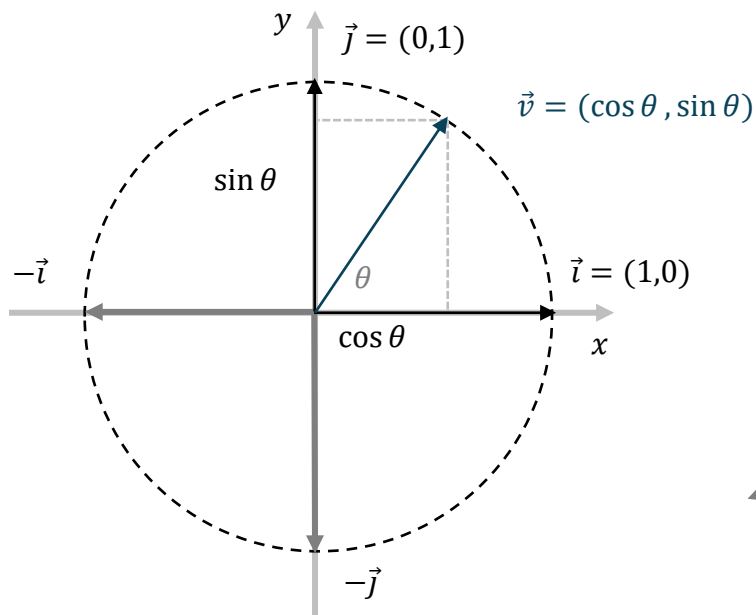
Skalarprodukt und Halbräume

Wenn das **Skalarprodukt** $\vec{w} \cdot \vec{v}$ zweier Vektoren $\vec{w}$ und $\vec{v}$ **positiv** ist, liegen beide Vektoren im selben **Halbraum**. Diese Eigenschaft ist sehr nützlich, um bspw. für eine Oberfläche, die durch eine Normale $\vec{n}$ repräsentiert wird, zu entscheiden, ob ein Lichtstrahl darauf Einfluss hat und berücksichtigt werden muss oder nicht.



Einheitskreis und Normierung eines Vektors

Nützlich in diesem Zusammenhang ist auch die Bedeutung des [Einheitskreises](#), der einen Radius von 1 aufweist und den Achsen. Auch die [Normierung](#) eines Vektors ist hier eine relevante Operation.





Was könnte bei der **Normierung eines Vektors** problematisch werden und wie gehen wir damit um?

Normierung eines Vektors

Der versierte Javaprogrammierer sieht sofort, dass sich diese Rückgabestruktur eines Wertes basierend auf einem booleschen Ausdruck auch mit dem einzigen dreistelligen Operator in Java ausdrücken lässt:

```
public static Vektor3D norm(Vektor3D vec) {  
    return (vec.isNullvector())  
        ? norm(new Vektor3D(rand.nextDouble(), rand.nextDouble(), rand.nextDouble()))  
        : new Vektor3D(div(vec, vec.length()));  
}
```

Java

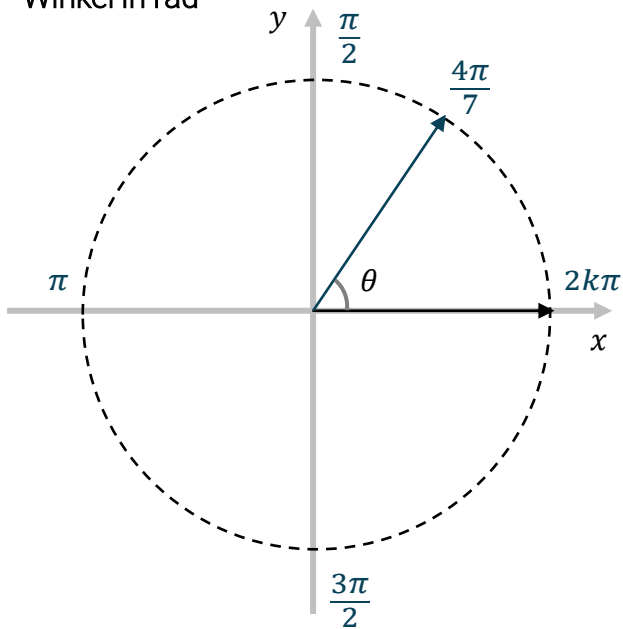
Welche Alternativen wären hier möglich?



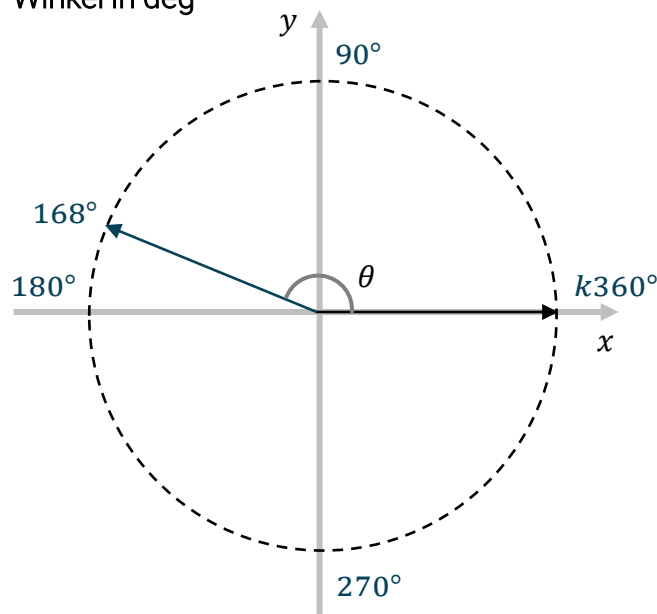
Winkelangaben in Bogenmaß oder Radiant

Die **Winkelangaben** beim Ablaufen des Kreises lassen sich als Bogenmaß (**Radiant**) oder in **Grad** angeben. Gelaufen wird immer in mathematisch positiver Richtung (gegen den Uhrzeigersinn).

Winkel in rad



Winkel in deg



Winkelangaben umwandeln

Die Winkelangaben lassen sich leicht ineinander umwandeln:

```
public static double radToDegree(double rad) {  
    return 180*rad / Math.PI;  
}  
  
public static double degreeToRad(double degree) {  
    return Math.PI*degree / 180;  
}
```

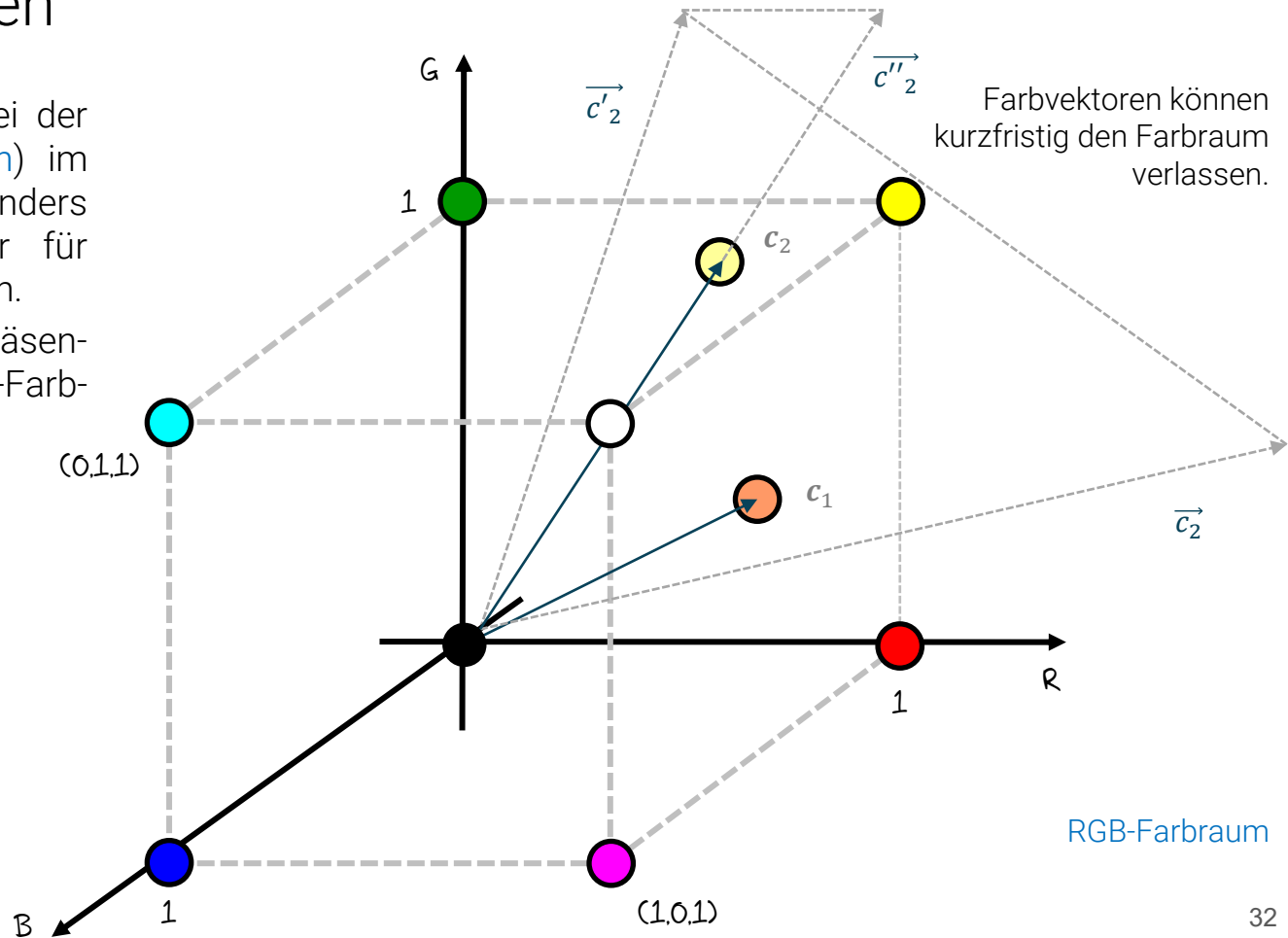
Java



Vektoren als Farben

Da sich Farben besonders bei der Multiplikation ([Farbmodulation](#)) im Vergleich zu Vektoren anders verhalten sollen, führen wir für Farben eine eigene Notation ein.

Mit $\mathbf{c}$, $\mathbf{d}$ werden Vektoren repräsentiert, die Farben des RGB-Farbraumes darstellen.

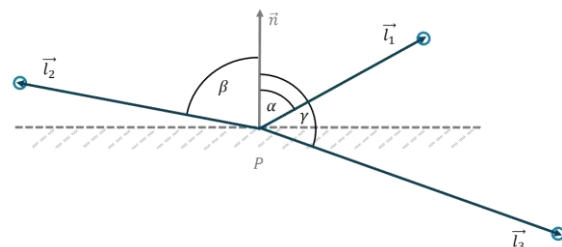


Vektoren liegen also in drei Varianten vor

Vektoren als Richtungen

$\vec{v}, \vec{w}$

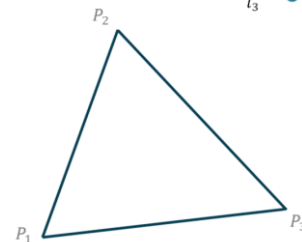
$$\cos \alpha = \frac{\vec{v}^T \vec{w}}{\|\vec{v}\| \cdot \|\vec{w}\|}$$



Vektoren als Positionen

L, P, Q

$$F = \frac{1}{2} \|(P_2 - P_1) \times (P_3 - P_1)\|$$



Vektoren als Farben

$\mathbf{c}, \mathbf{d}$

$$\mathbf{c} + \mathbf{d} = (c_R + d_R, c_G + d_G, c_B + d_B)$$

$$\mathbf{c} \cdot \mathbf{d} = (c_R \cdot d_R, c_G \cdot d_G, c_B \cdot d_B)$$



Modulation

Grundlagen der Vektorgeometrie

Key Takeaways



- Vektoren haben je nach Kontext unterschiedliche Bedeutungen: Richtungen, Positionen oder Farben
- Skalarprodukt liefert Winkelinformationen
- Kreuzprodukt liefert Flächenmaß und Orientierung
- Normierung ist zentral für Beleuchtungsmodelle
- Kosinusformel ist zentrale Formel der Computergrafik
- Clamp-Operationen verhindern numerische Artefakte

Fragestellungen zum Vorlesungsteil

Im Anschluss an diese Veranstaltung sollten Sie die folgenden Fragen sicher beantworten können:

- (1) Warum notieren und behandeln wir Vektoren, die Farben repräsentieren, anders als Vektoren, die beispielsweise Richtungen oder Positionen repräsentieren?
- (2) Warum hatte der Vulkan, auf dem der trigonometrische Gott der Winkel „Sohcahtoa“ lebt, den folgenden Namen erhalten: „Cotaosechacosho“?
- (3) Beweisen Sie folgendes Theorem: Gegeben seien zwei 3-dimensionale Vektoren $\vec{v}$ und $\vec{w}$, dann erfüllt das Kreuzprodukt die folgende Gleichung: $\|\vec{v} \times \vec{w}\| = \|\vec{v}\| \|\vec{w}\| \sin \alpha$.

Hinweis: Beginnen Sie zunächst damit, die linke Seite zu quadrieren und geeignet umzuformen. Es sollte folgender Term entstehen: $\|\vec{v}\|^2 \|\vec{w}\|^2 - (\vec{v}^T \vec{w})^2$. Die Auflösung des Kreuzprodukts zweier 3-dimensionaler Vektoren im rechtshändigen Koordinatensystem war ja gerade:

$$\vec{a} \times \vec{b} = \begin{bmatrix} a_2 b_3 - a_3 b_2 \\ a_3 b_1 - a_1 b_3 \\ a_1 b_2 - a_2 b_1 \end{bmatrix}$$

Anschließend können wir die Kosinusformel einsetzen und den $\cos$ -Term geschickt in $\sin$ überführen, da laut Pythagoras auch gilt $1 = \cos^2 \alpha + \sin^2 \alpha$.

Praktische Aufgaben

Die praktischen Aufgaben dienen zur Vertiefung des Stoffs und sollten selbstständig bearbeitet werden.

Aufgabe 1) [Java] Erklären Sie in einfachen Worten die Motivation hinter der Unterscheidung der Klassen Vektor2D bzw. Vektor3D und der Klasse LineareAlgebra. Was hat es in diesem Zusammenhang mit dem privaten Konstruktor auf sich?

Aufgabe 2) [Java] In der Klasse TriangleInterpolation im Package kapitel04/loesungen finden Sie vier unterschiedliche Lösungen für die baryzentrische Interpolation. Finden Sie durch geeignete Tests heraus, ob alle vier Lösungen korrekt sind. Überlegen Sie sich ein geeignetes Verfahren, um die Performance der vier Lösungen zu vergleichen.

Aufgabe 3) [Java] Verwenden Sie die baryzentrische Interpolation in einer interaktiven GUI in Java. Platzieren Sie drei Punkte (größere Kreise) mit der Maus auf den Bildschirm. Achten Sie darauf, dass die drei Punkte ein Dreieck aufspannen. Jeder Punkt repräsentiert einen unterschiedlichen Farbwert. Jetzt kann die Maus einen Punkt innerhalb des Dreiecks anklicken und es erscheint ein neuer Punkt (größerer Kreis), der mit dem durch die baryzentrische Interpolation ermittelten Farbwert gefüllt ist. Punkte, die außerhalb des Dreiecks angeklickt werden sollen entsprechend visualisiert werden.



Vielen Dank für die Aufmerksamkeit